

The SPV-to-SAGA invoice pipeline

How incoming e-Factura invoices get archived, matched to a contract, read, classified, and handed to SAGA — with an accountant kept in the loop wherever the machine shouldn't decide alone.

Prepared for Dev team **Source** Product workflow walkthrough

Status Draft v1 — open questions flagged inline

PART I

Concept note

What this is

An invoice that lands in a client's ANAF SPV inbox should be able to walk itself, unattended, into a state that's ready to book in SAGA — archived, checked against the right contract, deduplicated, fully read, and tagged with the account, VAT treatment and deductibility each line needs. The machine does the reading and the reasoning; the accountant is only pulled in at the points where a real judgment call has to be made, and every one of those points surfaces as a single decision, not a form to fill in.

The two halves of this document give the dev team a shared picture: what the pipeline is *for* (this part), and how it's built (Part II).

Who's involved

- **The system** — pulls invoices, files them, reads them, and proposes every classification. Runs unattended by default.
- **The accountant** — the only human in this flow. Sees a task only when the system is genuinely unsure (which contract, whether a contract exists at all, an unusual classification) and can always correct a proposal the system made confidently.

Nothing here assumes a bookkeeper types data in. The accountant's time is spent adjudicating the handful of cases per batch that need a person — everything else should never reach them.

The pipeline, end to end

1 Download from SPV

Pull new invoices from the client's ANAF e-Factura inbox.

2 File in the client's archive

Every invoice is stored under its client before anything else happens to it.

3 Match to a contract **ACCOUNTANT MAY BE ASKED**

Find the contract this invoice belongs to and check it's consistent with it. One clean match proceeds alone; several candidates or none get a decision request.

4 Check for duplicates

An invoice already on file is recognized and not re-processed.

5 Read the header

Doc number, supplier, dates, amounts, VAT, exchange rate, SPV upload id.

6 Read every line

Item, unit, VAT, quantity, unit price, value, total, notes — per line.

7 Classify every line **ACCOUNTANT MAY BE ASKED**

Account code, VAT rate, and deductibility, each with its legal basis. Confident calls pass through; uncertain ones wait for a look.

8 Export to SAGA

An XML file SAGA can import directly, fully populated.

Where humans stay in the loop — and why only there

Three moments in the flow are allowed to stop and ask, because each one is a judgment call the system cannot safely make alone:

- **Several contracts match the client.** The system picks its best guess and asks for a one-tap confirmation rather than silently committing.
- **No contract is on file.** The system can't invent one — it asks for the contract, once, and holds the invoice until it arrives.
- **A line's classification is uncertain** (novel item, an unusual expense type, a VAT rate that doesn't match what the category expects). The system proposes its best answer and flags it rather than guessing silently.

Everywhere else — filing, deduping, reading fields, exporting — is mechanical and should never interrupt anyone. That split is the whole design principle: automate the reading, escalate the judging.

What "done" looks like

- An invoice with a clean, unambiguous contract match and confident classifications reaches SAGA with zero human touches.
- An invoice that does need a human gets exactly one clear question, not a review of the whole document.
- Every automatic decision — which contract, which account, which VAT rate, which deductibility — can be explained after the fact: what fired, and on what legal basis.

PART II

Technical note

Architecture shape

Each invoice moves through a small, explicit state machine rather than a single script, because two of its steps end in a wait-for-a-human state that can last hours or days:

DOWNLOADED → ARCHIVED → MATCHING → (AWAITING_CONTRACT | AWAITING_MATCH_CONFIRM) → DEDUPE_CHECKED → HEADER_READ → LINES_READ → CLASSIFIED → (AWAITING_REVIEW) → EXPORTED

A durable workflow engine (Temporal, AWS Step Functions, or — if the team wants to start lighter — a status column on the invoice row plus a queue worker per transition) fits this better than a monolithic batch job, since a run can legitimately sit paused for days waiting on an accountant.

RECOMMENDATION

Treat the accountant-facing "pop-ups" as a **task inbox**, not a modal: each stalled invoice becomes a row (type, context, system's proposed answer, accept/override actions), so one accountant can clear a batch instead of being interrupted per invoice.

Core data model

Client

CUI, name, SPV credentials/certificate reference.

Contract

client_id, currency, value, unit type, period start/end, payment terms, source document.

Invoice

client_id, contract_id (nullable), spv_upload_id, header fields, dedupe hash, status, raw XML reference.

Invoiceline

invoice_id, line fields, cont_contabil, cota_tva, deductibilitate, confidence, reviewed_by/at.

ValidationTask

invoice_id, type, proposed answer, status, assignee, resolution.

ClassificationRule

rule_type, match pattern, result value, legal basis, effective_from/to, version.

Stage-by-stage spec

1 SPV ingestion

Pull new e-Factura messages from ANAF's SPV on a schedule (or as soon as SPV notifies of a new message, if that mode is available), authenticated with the client's qualified digital certificate. Each downloaded invoice arrives as structured XML (UBL 2.1 or CII, per the EN16931 e-invoicing standard) — this is a real advantage over a paper/PDF pipeline: no OCR is needed, only XML parsing.

STORE PER DOWNLOAD

ANAF's own message/upload id, download timestamp, and the raw XML — before any parsing happens, so a parsing bug never loses the source document.

CONFIRM BEFORE BUILDING

ANAF's SPV API surface (endpoints, auth flow, message-listing model) has changed before and may again — verify the exact contract against ANAF's current technical documentation rather than this note.

2 Client archive

Persist the raw XML immutably (object storage), keyed by client and SPV id, with metadata indexed in the database for lookup. Romanian bookkeeping law imposes long retention on financial records — the archive should be append-only / non-destructive by design, not just by policy.

3 Contract matching

Look up contracts for the invoice's supplier/client. Then:

- **Exactly one contract** — check currency, amount consistency, unit type, period coverage, and payment method against the invoice; if consistent, proceed automatically.
- **Multiple contracts** — score each candidate on the same fields, take the best-scoring one as a proposal, and open a `ValidationTask` (type `contract_match`) for a one-tap confirm/reassign.
- **No contract** — open a `ValidationTask` (type `no_contract`) and hold the invoice in `AWAITING_CONTRACT` until one is supplied.

PRODUCT DECISION NEEDED

The source workflow doesn't say what happens when the single matching contract's fields *don't* line up (wrong currency, expired period, etc.). Decide explicitly whether that auto-flags for review or is treated as a harder rejection — don't let engineering default it silently.

4 Duplicate check

ANAF's upload/message id is already a strong primary key — reject re-downloads of an id already on file. As a second layer against duplicates entering by another path (manual entry, re-upload under a new SPV id), compute a content hash from supplier CUI + doc number + issue date + total and check it against the archive.

5 Header extraction

A deterministic XML → field mapping (confirm exact element paths against ANAF's published UBL/CII XSD):

FIELD	MEANING
Nr. doc	Invoice document number
Furnizor	Supplier name / CUI
Data issue	Issue date
Data scadenta	Due date
Valoare	Net value
TVA	VAT amount
Total	Gross total
Neachitat	Amount outstanding
Adaos	Markup, where applicable
Curs ref.	Reference exchange rate
Id incarcare SPV	ANAF's SPV upload id — also the dedupe key

6

Line extraction

Parse the repeating invoice-line elements into one `InvoiceLine` row each:

FIELD	MEANING
Denumire articol/serviciu	Item or service description
UM	Unit of measure
TVA	VAT rate stated on the line
Cantitate	Quantity
Pret unitar	Unit price
Valoare	Line net value
TVA	Line VAT amount
Total	Line gross total
Informatii suplimentare	Free-text notes on the line

Line classification — three rule engines

Each line gets three answers, each backed by a versioned, accountant-maintained rule table rather than logic hardcoded into the pipeline — the legal texts define categories and limits, they don't hand you a lookup table:

A — CONT CONTABIL

Ordin 1802/2014 Cap. 14 defines what each account in the general chart of accounts *is* — it doesn't say which account a given purchased item maps to. That mapping is real business classification: `item/service → expense category → account code`. Build it as a keyword/pattern rule table the accountant owns and edits, with the pipeline proposing a category and confidence score (simple keyword rules first; a similarity/embedding model later if volume justifies it). Below a confidence threshold, the line goes to review rather than posting on a guess.

B — COTA TVA

Art. 291 alin. (1)–(2) sets the standard rate and the reduced rates for enumerated categories of goods and services. Since the invoice line already states its own VAT rate (step 6), treat this less as "compute the rate" and more as a **cross-check**: does the stated rate match what Art. 291 says this category should carry? A mismatch is flagged — it's often a supplier error, and worth a human eye either way.

C — DEDUCTIBILITATE

Art. 25 splits expenses into fully deductible, capped/partial (e.g. certain vehicle-related costs, protocol expenses within a ceiling), and non-deductible (e.g. fines, out-of-scope sponsorship). Same shape as (a): a category → deductibility-class rule table, versioned by effective date since ceilings and categories shift with each fiscal year's Codul Fiscal amendments.

OWN THIS JOINTLY WITH ACCOUNTING, NOT ENGINEERING ALONE

The three rule tables encode real tax judgment. Build them with the client's accountants from day one, keep every rule's legal citation attached, and make every automatic classification editable inline before export — feed corrections back into the table instead of re-deciding the same case forever.

8 SAGA export

Generate the XML SAGA expects for import, populated from the header, lines, and their three classification fields. Model the domain internally first (a plain invoice + lines object) and keep the SAGA XML writer as a thin, swappable adapter over it — useful discipline in itself, and it means a second target format (or a different client's ERP) doesn't touch the extraction or classification logic.

CONFIRM BEFORE BUILDING

Get SAGA's official import schema from Saga Software and validate one hand-built sample file against a real SAGA import before writing the generator. In particular, check whether SAGA expects raw Ordin 1802 account numbers or a firm-specific sub-account scheme — the two can differ.

Cross-cutting concerns

- **Audit trail.** Every automatic decision and every human override is logged immutably — actor, timestamp, before/after value. This is both a compliance requirement and the feedback signal for improving the rule tables.
- **Security.** SPV access rides on a qualified digital certificate per client — plan for certificate storage and rotation. Financial documents are encrypted at rest. Access is scoped per client (CUI), since one firm's login serves many clients' books.
- **Rule administration.** The three classification tables and the contract-matching tolerances need a small admin surface accountants can edit without a deploy — this is arguably as important to build well as the pipeline itself.
- **Idempotency.** Every stage should be safely re-runnable on the same invoice (retries, redeploys) without double-filing, double-matching, or double-exporting.

Open questions before implementation starts

01 ANAF SPV API contract

Confirm current endpoints, auth flow, and message model directly against ANAF's technical docs.

02 SAGA's exact import schema

Obtain and validate against Saga Software's official documentation before building the exporter.

03 Single-contract mismatch behavior

Not specified in the source workflow — needs an explicit product decision (auto-flag vs. reject).

04 Match tolerances

What counts as "everything makes sense" for amount variance, contract-period grace, etc. — needs numbers, not a vibe.

05 Rule-table ownership

Who on the accounting side builds and maintains the account/VAT/deductibility mapping tables, and how often.

Draft prepared for internal engineering discussion – not a substitute for review by qualified tax counsel before the classification rules go live.